International Journal of Information Security (2026) 25:187
https://doi.org/10.1007/s10207-026-01346-7

REGULAR CONTRIBUTION

A hybrid framework for web applications security vulnerabilities detection

James A. Stewart[1] · Gheida Shahrour[1] · Amro Al-Said Ahmad[1]

Received: 8 July 2026 / Accepted: 7 October 2026
© The Author(s) 2026

Abstract

The threat landscape for modern web applications is constantly changing, and traditional security approaches struggle with comprehensive vulnerability detection. Black-box testing, although widely adopted, suffers from limited path coverage and static payload limitations. Model-Based Testing (MBT) provides systematic exploration but requires substantial effort and may miss implementation-specific vulnerabilities. This research introduces and validates a hybrid methodology that integrates black-box testing and MBT through a six-level enhancement framework incorporating authentication mechanisms, client-side crawling, custom payload optimisation, and scope refinement. To demonstrate practical implementation and validate theoretical foundations, the methodology was applied as a case study using OWASP Zed Attack Proxy (ZAP) against OWASP Juice Shop, with manual verification of 3 discovered vulnerabilities and detailed statistical analysis across multiple test iterations. The results demonstrate substantial improvements in vulnerability detection effectiveness, with strong statistical evidence. The case study implementation revealed critical tool reliability issues requiring organisational attention, including systematic documentation of interface reporting discrepancies within automated scanners affecting vulnerability count accuracy, necessitating multi-verification approaches for organisational security assessment. The research contributes (1) a systematic integration framework, designed around tool-agnostic principles and validated through a single-tool case study, enabling incremental adoption across diverse security testing platforms, (2) a comprehensive empirical validation methodology with statistical rigour, and (3) practical implementation guidance including reproducible automation scripts adaptable to various organisational contexts. The framework's design principles and progressive enhancement approach provide a foundation for broader adoption beyond the specific ZAP implementation demonstrated in this validation study. The sixth level achieved a detection rate of 23.7 critical vulnerabilities compared to 2 critical vulnerabilities in baseline testing, representing a 1085% increase (note: based on $n = 3$ iterations per configuration; reported as exploratory rather than confirmatory). Total vulnerability detection increased from 583 to 4427, a 659% improvement. This demonstrates the cumulative effectiveness of systematic enhancement approaches in automated security testing within the single-application case study evaluated here.

Keywords Hybrid testing methodology · Web application security · Vulnerability detection · Model-based testing · Black-box testing · OWASP · Security assessment framework

1 Introduction

The exponential growth of web applications has fundamentally transformed the digital landscape, creating unprecedented opportunities alongside significant security challenges [1]. Modern web applications serve critical functions across financial services, healthcare, e-commerce, and government sectors, making their security critical to organisational operations and public safety [2]. This digital transformation has simultaneously expanded the threat landscape, with attackers ranging from opportunistic individuals to sophisticated state-sponsored entities employing diverse

✉ Amro Al-Said Ahmad
a.m.al-said.ahmad@keele.ac.uk

James A. Stewart
x6k18@students.keele.ac.uk

Gheida Shahrour
g.shahrour@keele.ac.uk

[1] School of Computer Science and Mathematics, Keele University, Staffordshire, UK

Published online: 10 November 2026

attack vectors including SQL Injection (SQLi), Cross-Site Scripting (XSS), and emerging API-based vulnerabilities. The persistent evolution of attack techniques has created a continuous "arms race" between vulnerability discovery and security tool development. Traditional security testing approaches face significant challenges in addressing modern web application architectures, particularly single-page applications (SPAs) with dynamic content generation, complex authentication mechanisms, and sophisticated defensive measures [3]. This technological evolution has outpaced the capabilities of conventional testing methodologies, creating critical gaps in vulnerability detection effectiveness [4].

1.1 Problem statement and research motivation

Current web application security testing approaches typically fall into two primary categories, each with distinct advantages and significant limitations when used in isolation. Black-box testing examines applications without internal knowledge of system architecture, simulating external attacker behaviour [5]. Whilst widely adopted due to accessibility and ease of implementation, these methods frequently fail to achieve comprehensive path coverage in modern web applications [6]. Analysis of baseline automated scanning reveals coverage limitations, accessing only 7 of 22 available application pages (32% coverage) and detecting few critical vulnerabilities (CV) despite the presence of numerous vulnerabilities in the test environment. Model-Based Testing (MBT) provides systematic exploration capabilities through formal models of application behaviour, offering comprehensive coverage and structured test execution [7].

However, MBT approaches often require substantial effort to construct and maintain behavioural models, specialised modelling expertise, and may create additional barriers to adoption [8–10]. Moreover, because testing is derived from abstract representations of expected behaviour, implementation-specific defects and vulnerabilities emerging from runtime interactions, configuration errors, or coding flaws may remain undetected when such aspects are not adequately captured in the model [10–12]. While black-box testing offers greater accessibility by operating without such models, it correspondingly sacrifices the systematic behavioural coverage that MBT provides. Therefore, the fundamental challenge lies in the inherent trade-off between accessibility and comprehensiveness. Organisations require security testing approaches that combine the practical advantages of black-box testing with the systematic exploration capabilities of MBT approaches, without requiring complete replacement of established tools and practices [13].

1.2 Research gaps and contributions

This research addresses critical gaps in existing web application security testing approaches through systematic investigation and empirical validation. The key contributions are organised around the identified gaps and corresponding solutions:

- Current approaches employ either black-box or MBT methodologies in isolation, failing to leverage complementary strengths systematically. We developed and validated a systematic six-level progressive enhancement framework that integrates black-box and MBT approaches, empirically validated through comprehensive experimentation with statistical analysis across multiple iterations [14].
- Limited systematic investigation of automated scanner reliability despite widespread organisational dependence on these systems for critical security assessment. We provide a comprehensive evaluation demonstrating statistically significant improvements in vulnerability detection effectiveness, coupled with critical identification of automated scanner reliability issues affecting organisational security assessment practices, including documentation of up to 200–300% discrepancies in vulnerability count accuracy [15].
- Existing research lacks comprehensive statistical validation of detection improvements, limiting confidence in reported effectiveness claims. We built a systematic manual verification protocol that validates automated findings through detailed exploitation techniques, achieving successful verification across the vulnerability types included in the sample, including timing-based SQLi attacks with quantified timing differentials [16].
- Theoretical frameworks exist for enhanced security testing, but practical implementation guidance enabling organisations to improve existing workflows remains limited. To address this gap, we implemented a complete reproducible framework including custom automation scripts, detailed configuration protocols, and comprehensive troubleshooting documentation enabling immediate organisational adoption with publicly available source code [17].
- Absence of systematic documentation of tool reliability issues and their implications for organisational security practices. We provide comprehensive documentation of significant inconsistencies in automated vulnerability reporting interfaces, revealing critical reliability concerns and providing evidence-based mitigation strategies for organisational security assessment [18].

1.3 Hypothesis and research questions

Our research hypothesis is that the systematic integration of black-box and MBT methodologies through a progressive enhancement framework will demonstrate statistically significant improvements in web application vulnerability detection compared to traditional standalone approaches, whilst revealing critical reliability issues in automated scanning tools that require organisational mitigation strategies. This research addresses the following research questions:

- **RQ1**: How can black-box and MBT approaches be systematically integrated to achieve statistically significant improvements in vulnerability detection whilst maintaining practical accessibility?
- **RQ2**: What specific integration strategies provide measurable improvements in testing effectiveness, and how do these improvements compound across enhancement levels?
- **RQ3**: What reliability challenges exist in automated vulnerability detection tools, and what are the implications for organisational security assessment practices?
- **RQ4**: How can comprehensive manual verification protocols validate the accuracy of automated detection improvements and confirm practical exploitability of discovered vulnerabilities?
- **RQ5**: What implementation guidance and automation frameworks are required to enable organisational adoption of hybrid testing methodologies?

The remainder of this paper is organized as follows: Sect. 2 reviews the web security threat landscape and establishes gaps in traditional testing approaches. Sect. 3 delineates the proposed hybrid framework, experimental setup, and custom automation scripts. Sect. 4 presents the quantitative empirical findings and performance analysis. Sect. 5 discusses the methodology's strategic implications, limitations, and future research avenues. Finally, Sect. 6 concludes the paper by summarising key contributions and recommendations.

2 Related work and background

2.1 Web application security threat landscape

The modern web application threat landscape encompasses a broad spectrum of attack vectors that have evolved alongside technological advancement [19]. XSS attacks enable malicious script injection into web pages viewed by users, potentially leading to session hijacking, credential theft, and unauthorised data access [20]. SQLi vulnerabilities exploit inadequate input validation in database query construction, allowing attackers to manipulate database operations and potentially access sensitive information [21]. Contemporary threat intelligence indicates that web application vulnerabilities continue to represent the majority of successful cyberattacks, with OWASP Top 10 vulnerabilities consistently appearing in production systems despite widespread awareness and available mitigation techniques [22]. This persistence suggests fundamental limitations in current security testing and validation approaches rather than simply implementation oversights [6].

2.2 Traditional testing approaches: strengths and limitations

2.2.1 Black-box testing approaches

Black-box testing methodologies examine applications from an external perspective without internal system knowledge, simulating the approach an external attacker would employ. Empirical analyses of baseline black-box testing reveals significant quantitative limitations, particularly in terms of coverage where baseline automated scanning achieved access to only 7 of 22 available application pages (32% coverage), with critical functionalities remaining inaccessible due to authentication barriers and complex navigation requirements [23].

Detection effectiveness shows standard black-box approaches detected only 2.0 ± 0.0 CV despite comprehensive vulnerability presence in the test environment, indicating substantial detection gaps consistent with prior observations regarding the limitations of automated black-box testing [24]. Recent benchmarking studies further confirm that state-of-the-art black-box vulnerability scanners continue to miss known vulnerabilities and exhibit substantial false-negative rates even under maximum-coverage configurations [23, 25]. Static payload constraints mean traditional approaches rely on predetermined payloads designed to trigger known vulnerabilities. Modern applications increasingly employ sophisticated input validation and sanitisation that effectively counter static attacks, leading to false negatives and missed vulnerabilities [26].

2.2.2 Model-based testing approaches

MBT approaches employ formal models of application behaviour to guide systematic test execution, offering complementary advantages to black-box methods [27]. Formal models enable systematic coverage of application states, transitions, and interaction patterns, thereby supporting comprehensive exploration of application functionality [28]. Structured test generation provides model-driven test case generation that offers reproducible, comprehensive testing protocols that can be adapted and refined over multiple iterations [29]. However, MBT approaches present significant practical limitations including substantial model creation and

maintenance overhead, specialised expertise requirements, and implementation gap issues in which abstract behavioural models may fail to capture runtime-specific system characteristics and evolving operational conditions [30, 31].

2.3 Comprehensive literature analysis

Recent research has addressed a wide range of challenges in web application security testing, with increasing emphasis on modern application architectures such as SPAs, API-driven systems, and the integration of security testing within Continuous Integration and Continuous Deployment (CI/CD) pipelines. The growing complexity of modern web applications has exposed limitations in traditional vulnerability assessment approaches, particularly regarding automated discovery, authentication handling, and comprehensive vulnerability coverage. Although significant advances have been made in automated vulnerability scanning, developing a reliable and comprehensive detection system that can effectively operate across diverse web applications remains a considerable challenge [32].

2.3.1 Commercial security testing platforms

Commercial vulnerability scanners represent a significant segment of the security testing landscape, offering enterprise-grade capabilities with varying levels of effectiveness and automation. Burp Suite Professional, developed by PortSwigger, provides comprehensive web application security testing through sophisticated crawling engines, extensive payload libraries, proxy-based analysis, and advanced authentication handling capabilities [33].

Previous studies have consistently identified Burp Suite as one of the most effective tools for detecting common web application vulnerabilities such as SQLi, XSS, and insecure authentication mechanisms. However, recent evaluations have shown that scanner effectiveness remains highly dependent on scan configuration, authentication setup, crawler performance, and the complexity of the target application. Consequently, detection accuracy can vary significantly depending on how effectively the scanner is configured and the characteristics of the application under assessment [34]. Nessus, developed by Tenable, primarily focuses on infrastructure security assessment, configuration auditing, and vulnerability management. While Nessus provides strong coverage for known vulnerabilities, outdated software, and system misconfigurations, it is generally less capable of identifying application-layer vulnerabilities than specialised web application scanners.

Dedicated web application security testing tools typically provide broader coverage of vulnerabilities such as SQLi, XSS, and business logic weaknesses when compared with general-purpose vulnerability assessment platforms [35]. Rapid7's AppSpider provides automated dynamic security testing for modern web applications and includes support for JavaScript-heavy applications, APIs, and modern authentication mechanisms. Nevertheless, current research highlights that business logic vulnerabilities remain particularly challenging to detect using automated approaches because successful exploitation often depends on application-specific workflows, contextual knowledge, and multi-step interactions. These limitations affect most automated scanning solutions, including AppSpider, as they primarily rely on dynamic analysis techniques and predefined testing rules.

2.3.2 Open-source security testing frameworks

Open-source security testing frameworks provide accessible alternatives to commercial solutions whilst enabling extensive customisation and community-driven development. OWASP ZAP is one of the most widely adopted open-source web application security scanners, offering automated vulnerability detection, proxy-based analysis, API testing capabilities, and active scanning functionality. Its strong community support and continuous development have made it a popular choice for both academic research and industrial security assessments. The Web Application Attack and Audit Framework (W3AF) provides a Python-based, extensible architecture for web application security testing through a comprehensive plug-in framework supporting vulnerability discovery, exploitation, and reporting. Whilst extensibility remains one of its principal strengths, modern web application security research has highlighted the increasing difficulty automated scanners face when testing JavaScript-heavy, stateful, and API-driven applications. Complex user workflows, asynchronous content loading, and state-dependent navigation frequently reduce crawler effectiveness and limit comprehensive attack surface discovery [36]. Nikto is a lightweight web server scanner primarily focused on identifying outdated software versions, insecure server configurations, known server-side vulnerabilities, and information disclosure issues [37]. Although Nikto provides effective baseline server security assessment capabilities, its effectiveness is limited when assessing modern web application vulnerabilities that require interaction with dynamic content, client-side logic, authentication workflows, and business processes [37].

2.3.3 Hybrid and model-based testing research

Quantitative security evaluation research by Wen et al. [1] developed frameworks mapping OWASP Application Security Verification Standard (ASVS) to Common Weakness Enumeration (CWE), emphasising the need for automated evaluation approaches and real-world application validation [1]. This research highlighted critical gaps between theo-

retical security models and practical detection capabilities, motivating systematic approaches to validation and measurement. MBT vulnerability management research by Kenner [13] proposed semi-automated frameworks for vulnerability analysis across development lifecycle stages, implementing formal verification techniques for security property validation[38]. However, the research lacked specialisation for specific vulnerability types and implementation-specific detection, limiting practical applicability in complex production environments. The theoretical framework provided a foundation for systematic security assessment but required substantial adaptation for practical deployment.

Automated model-based vulnerability testing research by Lebeau et al. [14] adapted MBT for vulnerability detection through negative test cases, developing systematic approaches for security test generation and execution. The research demonstrated promising theoretical foundations but highlighted significant scalability challenges and identified the need for generic, reusable approaches that could be adapted across diverse application architectures. Implementation complexity and expertise requirements limited broader adoption beyond research environments. State-aware testing research by Doupé et al. [24] introduced advanced crawling and state management techniques for modern web applications, addressing limitations in traditional black-box scanning approaches [24]. The research demonstrated significant improvements in application coverage and vulnerability detection, though implementation remained platform-specific and lacked systematic integration with existing security testing workflows.

2.3.4 Reliability and effectiveness assessment studies

Scanner reliability assessment research by Fonseca et al. [39] investigated automated vulnerability scanner reliability through controlled benchmarking across multiple commercial and open-source platforms [40]. More recent evaluations extend these findings to contemporary scanning tools and DevSecOps-integrated environments, demonstrating persistent variation in detection accuracy, false-positive rates, and false-negative behaviour across both commercial and open-source scanners [23, 25]. This foundational study identified critical concerns about organisational dependence on automated tools, documenting significant variability in detection effectiveness and false positive rates that affect security assessment confidence. This foundational research established the need for systematic reliability evaluation and quality assurance frameworks. Comparative effectiveness research by Bau et al. [41] evaluated eleven commercial and open-source web vulnerability scanners using standardised test applications, revealing substantial differences in detection capabilities and reliability across platforms, a pattern that remains evident in recent comparative evaluations reporting significant variation in true-positive, false-positive, and false-negative performance across contemporary scanning tools [23]. The research demonstrated that no single scanner achieved comprehensive coverage, with detection rates varying from 15% to 67% across different vulnerability categories. This variation highlighted the need for multi-tool approaches and systematic validation protocols.

Performance evaluation research by Vieira et al. [42] conducted systematic benchmarking of web vulnerability scanners using real-world applications, identifying significant challenges in practical deployment including high false positive rates, limited coverage of complex authentication scenarios, and difficulty in handling modern web application architectures, challenges that remain evident in contemporary automated testing and CI/CD-oriented environments where coverage, industrial adoption, and realistic deployment validation continue to represent active research concerns [43, 44].

The research provided an empirical foundation for understanding scanner limitations and motivated development of enhanced detection approaches. Pellegrino et al. [45] investigated the prevalence of human-related security vulnerabilities in web applications, demonstrating that automated scanners consistently miss critical vulnerability categories related to business logic, authorisation, and complex authentication flows, a pattern corroborated by recent work showing that authorisation and business-logic vulnerabilities remain difficult for automated tools to detect without application-specific context [46, 47], and one that remains consistent with our own results in Sect. 4. This research highlighted fundamental limitations in automated detection approaches and motivated development of hybrid methodologies that combine automated scanning with systematic manual verification protocols.

2.3.5 Tool selection and justification for case study implementation

Given the diverse landscape of web application security testing tools, the selection of an appropriate platform for hybrid methodology validation requires careful consideration of multiple factors including accessibility, extensibility, community support, and research reproducibility requirements [48]. This research selected OWASP ZAP as the primary case study platform for implementing and validating the hybrid methodology based on several critical justifications.

- **Accessibility and cost considerations:** Unlike commercial platforms such as Burp Suite Professional or AppSpider that require substantial licensing investments (typically £399–£3,999 annually per user). Although free versions are available, they have limited capabili-

ties; ZAP, by contrast, provides comprehensive security testing capabilities through open-source licensing, eliminating financial barriers to research replication and organisational adoption. This accessibility ensures that methodology validation results remain applicable across diverse organisational contexts, from academic institutions to resource-constrained small enterprises [33].

- **Extensibility and customisation capabilities:** ZAP's plugin-based architecture and comprehensive API support enable systematic integration of custom automation scripts and behavioural modelling components essential for hybrid methodology implementation. Unlike closed-source commercial solutions that limit customisation options, ZAP's open architecture permits deep integration of MBT components whilst maintaining compatibility with existing organisational workflows.
- **Community support and development ecosystem:** As the flagship security testing project of the OWASP Foundation, ZAP benefits from extensive community contribution, continuous development, and comprehensive documentation that supports research reproducibility and long-term methodology sustainability. The active development community ensures ongoing platform evolution and addresses emerging web application architectures, providing a foundation for adapting and enhancing the methodology.
- **Research reproducibility and validation:** ZAP's open-source nature enables complete methodology reproduction, including exact configuration replication, custom script verification, and independent validation by research communities. Commercial platforms often restrict reproducibility through proprietary algorithms, licensing limitations, and version control constraints that impede scientific validation and peer review processes.
- **Baseline performance characteristics:** Empirical evaluation demonstrates that ZAP's baseline detection capabilities exhibit limitations consistent with broader automated scanning challenges documented in literature, including coverage constraints (32% application area access) and reliability issues (Coefficient of Variation (CoV) up to 57.7%) that provide a realistic foundation for methodology enhancement validation. These characteristics ensure that improvement measurements reflect genuine advancement rather than artificial baseline manipulation.
- **Industry adoption and practical relevance:** ZAP maintains significant adoption across industry contexts, with deployment in organisations ranging from startups to multinational corporations, ensuring that methodology validation results provide practical relevance for real-world security assessment improvement [49]. This adoption pattern contrasts with academic-only tools that may lack practical applicability or commercial-only solutions that limit accessibility.

The selection of ZAP as the case study platform thus provides an optimal balance between research requirements (reproducibility, extensibility, cost-effectiveness) and practical considerations (industry relevance, community support, accessibility), whilst ensuring that hybrid methodology validation demonstrates genuine advancement over established baseline capabilities rather than comparison against artificially constrained alternatives.

2.4 Identified gaps and research motivation

Analysis of existing literature and empirical baseline testing reveals critical gaps that motivate the hybrid methodology development. Integration gap analysis shows current approaches typically employ either black-box or MBT methodologies in isolation, with relatively limited research focusing on unified frameworks that combine behavioural modelling with runtime vulnerability discovery techniques [44, 50]. Reliability assessment gap analysis reveals limited systematic investigation of automated scanner reliability despite widespread organisational dependence on these systems for critical security assessment [51]. Practical implementation gap analysis demonstrates that whilst theoretical frameworks exist for enhanced security testing, practical implementation guidance enabling organisations to improve existing workflows remains limited [52]. Statistical validation gap analysis shows most existing research lacks a comprehensive statistical analysis of detection improvements, limiting confidence in reported effectiveness claims [53]. These gaps collectively indicate the need for hybrid methodologies that can enhance existing tool capabilities whilst maintaining practical accessibility and providing rigorous empirical validation of effectiveness improvements.

3 Methodology

The methodology began with manual exploration of the target web application, OWASP Juice Shop, [54] to identify critical UI sequences and elements required for accessing deeper paths. These findings were first captured in a state-transition model using Draw.io. This model was then used to create the Selenium scripts and were then executed within ZAP's scripting framework. The scripts were configured as Selenium scripts using the ECMAScript engine, then named, enabled and saved within ZAP. The testing environment utilised non-default ZAP settings including a scan starting point of "localhost:3000" and an "Authentication Test" context with user "jim@juice-sh.op" and password "test123"

(password was bypassed due to an email vulnerability). Spider scan parameters included a maximum depth of 10, AJAX Spider and Client Spider using the Chrome browser with a maximum of 3 windows, Client Spider page load times of 10 s, and a custom Active Scan policy restricted to Buffer Overflow, XSS (Persistent and Reflected) variants, and SQLi variants.

The custom scripts automated drop-down navigation and complex UI interactions unreachable by standard scans. The custom scripts used CSS selectors to identify the buttons required to navigate through the webpages, and once identified, would click these buttons to crawl deeper [55]. ChromeDriver would execute these custom scripts on the browser and allow it to be automated [56]. Each test involved enabling the script to drive the interactions, followed by sequential executions of Spider, AJAX Spider, and Active Scan. Security alerts were recorded and saved in Excel spreadsheets after each run, with scripts re-enabled if necessary between test phases. Path coverage was measured by counting unique URLs discovered during scanning. Each test configuration was repeated three times to calculate averages and ensure result reliability.

3.1 Hybrid methodology framework

This research introduces a systematic hybrid methodology that integrates black-box and MBT through a structured progression of enhancement capabilities, designed for empirical validation and practical organisational adoption [57]. The hybrid methodology illustrated in Fig. 1, incorporates six levels to enhance the vulnerability detection in web applications systematically.

3.1.1 Progressive enhancement framework

The methodology consists of six progressive enhancement levels, each stage extends and depends on the capabilities introduced at the previous level. This ensures that functionality is validated before adding additional complexity to the tests. Before Level 1, manual exploration was performed to build the state-transition model of the application, following a four-step procedure intended to generalise to other target applications:

(i) **Surface enumeration**: manually navigate the application as an unauthenticated user, recording every reachable page, form, and navigation control as a candidate state.
(ii) **Transition capture**: for each state, record the actions (clicks, form submissions, drop-down selections) that lead to other states, including authentication-gated transitions identified but not yet accessible.
(iii) **Model construction**: encode states as nodes and actions as labelled edges in Draw.io, distinguishing pre-authentication and post-authentication subgraphs.
(iv) **Validation**: traverse every recorded path in the live application a second time, in a random order, to confirm that all transitions and elements matched observed behaviour; discrepancies were corrected before the model was used to design the Selenium scripts (Sect. 3.3).

This procedure is deliberately tool-agnostic: steps (i)–(iii) require only manual browsing and a diagramming tool, and step (iv) requires only the target application itself, so the same processes can be applied to other web applications under test. With the state-transition model in place, we applied the six progressive enhancement levels shown in Fig. 1 to detect vulnerabilities in the target application:

(L1) This establishes the baseline using black-box testing with traditional automated scanning under minimal configuration. Level 1 defines the reference performance metrics to be used for comparison with all subsequent levels. This baseline test also identified the coverage limitations, including the inability to authenticate which restricted access to deeper pages. This led to the introduction of authentication in Level 2.
(L2) This builds on Level 1 with the introduction of authentication integration, enabling access to protected areas of the application that were unreachable in Level 1 [58].
(L3) Level 3 addresses the remaining coverage limitations observed after authentication integration by introducing dynamic content discovery. Crawling capabilities for SPAs and dynamic content generation are improved to address the limitations present in Levels 1 and 2.
(L4) Level 4 combines the capabilities of authentication and dynamic discovery into one unified process. This level enabled interactions with authenticated sessions whilst exploring dynamically generated content allowing for assessment of their combined effect.
(L5) Builds on the integrated framework by introducing behavioural modelling through custom scripts that simulate realistic user interactions. This enables deeper exploration of complex application workflows beyond automated crawling alone.
(L6) This represents full integration of all previous enhancements, combining baseline scanning, authentication, dynamic discovery and behavioural modelling into a single optimised framework with improved resource allocation and performance tuning.

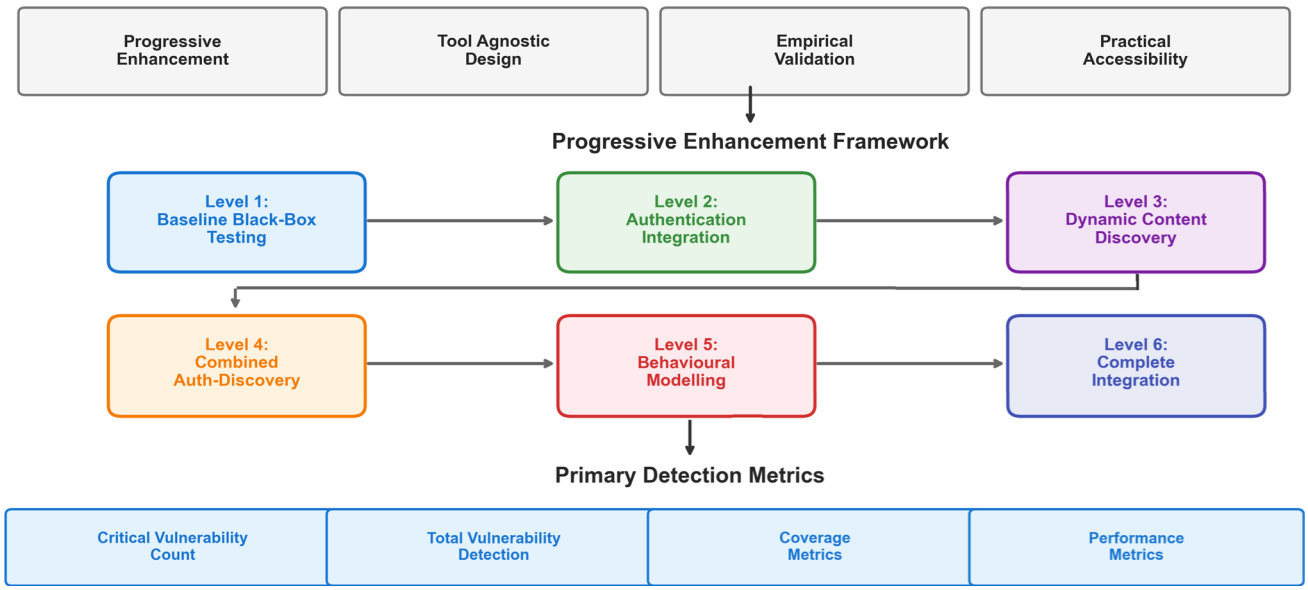


Fig. 1 Hybrid methodology framework

Finally, to validate whether the identified vulnerabilities were genuine, Postman was used to manually repeat the flagged vulnerabilities [59]. To enhance reproducibility the scripts were written, including all experimental configuration publicly available via GitHub at: https://github.com/OraclePark/ZAP-Automated-Scripts/tree/main.

3.1.2 Core design principles

Progressive enhancement ensures each level builds systematically upon previous capabilities, enabling incremental adoption and measurable improvement assessment. Tool agnostic design provides implementation guidance adaptable across different security testing platforms without vendor lock-in. Empirical validation ensures each enhancement level provides measurable improvements with statistical analysis enabling confidence assessment and optimisation. Practical accessibility maintains accessibility advantages of traditional black-box testing whilst systematically incorporating advanced capabilities.

3.2 Experimental design and implementation

3.2.1 Platform and target selection

OWASP ZAP was selected based on accessibility, extensibility, and community adoption factors enabling research reproducibility and practical relevance. OWASP Juice Shop was selected as the target application for comprehensive vulnerability coverage, modern SPA architecture, and controlled testing environment requirements. Infrastructure configuration employed Docker containerisation using Docker Desktop v4.15.0 with the OWASP Juice Shop container (bkimminich/juice-shop:latest) which was allocated 4GB of RAM and 2 CPU cores for consistent testing conditions. Local deployment on port 3000 eliminated service interruption issues encountered with hosted versions at demo.owasp-juice.shop [54].

3.2.2 Workload characterisation

To contextualise the detection surface presented to each test configuration, Table 1 reports the source composition of the deployed OWASP Juice Shop instance, and Table 2 reports the taxonomy, severity, and count of intentionally embedded vulnerabilities.

A single target application was used in this validation study by design: the research goal (RQ1–RQ5) was to establish, with tight experimental control and repeated iterations, whether the proposed six-level framework itself produces measurable improvements (not to benchmark absolute detection rates across the web application landscape). A multi-stack benchmark (e.g. adding OWASP WebGoat, which uses a Java/Spring stack rather than Juice Shop's Node.js/Angular SPA architecture) would strengthen external validity, but was outside the resource and time scope of this study; Sect. 5.5 identifies cross-application validation as the immediate next step, and the reproducible automation scripts (Sect. 3.3) are released specifically to enable that follow-on work by us or other groups. The workload characterisation in Table 2 is provided so readers can judge the representativeness of this specific target rather than assume it.

Table 1 OWASP juice shop source composition (deployed version)

Language	TypeScript	HTML	JavaScript	CSS/SCSS	Total
SLOC (approx.)	40,856	4066	21,407	2885	69,214

Table 2 Intentional vulnerability taxonomy by OWASP category and severity

OWASP category (approx.)	Critical	High	Medium	Low
Broken access control	1	1	4	5
Broken authentication	2	0	1	0
Cryptographic issues	1	1	1	1
Improper input validation	0	2	6	4
Injection	2	5	5	2
Insecure deserialisation	3	0	0	0
Miscellaneous	1	0	1	4
Security misconfiguration	2	0	0	3
Security through obscurity	1	1	1	0
Sensitive data exposure	3	5	1	7
Unvalidated redirects	0	1	0	1
Vulnerable components	6	2	1	0
XSS	1	3	2	3
Total	23	21	23	30

3.2.3 Statistical design and quality assurance

The multiple-iteration protocol required each test configuration to be executed three times to assess consistency and enable statistical analysis. This sample size provides adequate power for detecting large effect sizes whilst remaining practically feasible. The randomisation protocol involved randomising the test execution order to control for temporal effects and system state variations [60]. Configuration standardisation implemented strict protocols for consistent starting conditions, including complete sitemap clearing, standardised authentication state management, and systematic timing protocols [61].

3.3 Custom automation development

A critical component of the methodology was the development of automation scripts to address limitations in standard automated scanning approaches [62]. Script architecture comprised a custom automation solution developed using Selenium WebDriver with Chromium browser automation, providing reliable cross-platform compatibility and robust interaction capabilities [56, 57]. Key technical capabilities included modal dialogue management through automatic detection and handling of blocking pop-up dialogues preventing automated navigation, drop-down menu navigation through sophisticated simulation of multi-level drop-down menu interactions including hover events and timing considerations, CSS selector-based navigation through precision element targeting via manually identified CSS selectors enabling reliable interaction with complex interface elements, and state preservation through maintenance of application state across complex navigation sequences ensuring consistent testing conditions [56, 57].

3.3.1 Custom scripts

These scripts were developed to enhance the coverage of the scan by simulating user interactions that the standard Spider and AJAX Spider could not reach – chiefly multi-level drop-down navigation and modal dialogue dismissal. Listing 1 shows a representative excerpt (Version 6.0) that dismisses the welcome banner and drives the account drop-down menu using CSS-selector-based clicks.

Listing 1 Excerpt from Custom Script v6.0: modal dismissal and drop-down navigation.

```
// Attempt to close any popup using the ".close
// -dialog" selector
try {wait.until(
org.openqa.selenium.support.ui.ExpectedConditions.
visibilityOfElementLocated
(By.cssSelector(".close-dialog")));
var closeDialog = driver.
findElement(By.cssSelector
(".close-dialog"));
if (closeDialog.isDisplayed()) {
closeDialog.click();
logger("Closed popup using .close-dialog");
```

```
}
} catch (e) {
logger("Popup not visible or clickable: " + e);
}

// Open Account dropdown
waitAndClick(driver, wait, "#navbarAccount",
"Opened Account dropdown", true);
```

Earlier versions (1.0–2.0) established core functionality – the pop-up blocker and basic click simulation. Versions 3.0–5.0 iteratively expanded drop-down and multi-step navigation coverage. Version 6.0, used for the Test 5 and Test 6 results reported in Sect. 4, prioritised readability and simplified control flow over the earlier versions while preserving their coverage gains. Versions 7.0–8.0 were experimental attempts at a site-agnostic, modular rewrite and remain unfinished; they were not used in any reported test. The complete scripts for all versions are available at: https://github.com/OraclePark/ZAP-Automated-Scripts

3.4 Data collection and analysis protocol

The primary detection metrics comprised the critical-vulnerability count, representing the number of high-severity vulnerabilities detected across iterations, total vulnerability detection providing a comprehensive count including all severity levels for overall capability assessment, coverage metrics through quantitative measurement of application areas successfully explored including page counts and functional area access, and performance metrics through systematic measurement of scan duration and resource utilisation for cost-benefit analysis. The calculations were made using Sample Standard Deviation. Statistical analysis protocol included descriptive statistics with mean, standard deviation, and confidence intervals calculated for all primary metrics, inferential statistics through paired t-tests conducted for comparison between enhancement levels, effect size analysis with Cohen's d calculated to assess practical significance of improvements, and reliability assessment through coefficient of variation calculated to assess consistency across iterations.

4 Results and analysis

This section presents the empirical findings across all five research questions, beginning with the core integration results (RQ1–RQ2), followed by reliability findings (RQ3), manual verification (RQ4), and implementation guidance (RQ5).

4.1 RQ1: systematic integration of black-box and model-based testing

The primary research question examined whether black-box and MBT approaches could be systematically integrated to achieve statistically significant improvements in vulnerability detection whilst maintaining practical accessibility. The experimental validation demonstrates that the hybrid methodology produces transformative improvements over traditional standalone approaches.

4.1.1 Critical vulnerability detection enhancement

The progression from baseline black-box testing to complete hybrid integration shows substantial improvements in critical-vulnerability detection capabilities, as illustrated in Fig. 2. Baseline testing (Test 1) detected a consistent 2.0±0.0 CV across all iterations, representing the limited detection capability of standard automated scanning approaches. The introduction of authentication mechanisms (Test 2) maintained the same detection rate at 2.0±0.0 critical vulnerabilities, indicating that authentication alone did not improve critical-vulnerability detection in this configuration. The progressive enhancements began to show improvements in subsequent test levels. Test 3, incorporating client-side crawling, achieved 4.0±1.7 critical vulnerabilities, representing a 100% improvement over baseline conditions with statistical significance ($p<0.01$). This demonstrates that dynamic discovery mechanisms can substantially enhance detection capabilities beyond traditional authentication-based approaches. The combination of authentication and dynamic discovery (Test 4) achieved 7.7±1.2 critical vulnerabilities, representing a 285% improvement over baseline conditions with high statistical confidence ($p<0.001$). This finding demonstrates that individual enhancement components provide substantial benefits, but their systematic combination produces multiplicative rather than additive returns. Test 5 maintained similar performance levels at 7.7±1.5 critical vulnerabilities, also representing a 285% improvement with high statistical significance ($p<0.001$). The complete hybrid approach (Test 6) achieved the largest improvement, detecting 23.7±6.4 CV compared to the baseline 2.0±0.0 (a 1085% increase, $p < 0.001$), though this percentage reflects the very small baseline value and should be interpreted alongside the raw counts rather than in isolation.

This final configuration demonstrates the cumulative effect of integrating the enhancements. Achieving more than seven-fold improvement in critical-vulnerability detection rates compared to baseline black-box testing approaches. The comprehensive statistical summary in Table 3 provides the complete foundation for all claims made about progressive enhancement effects. The systematic progression from baseline through complete integration demonstrates distinct pat-

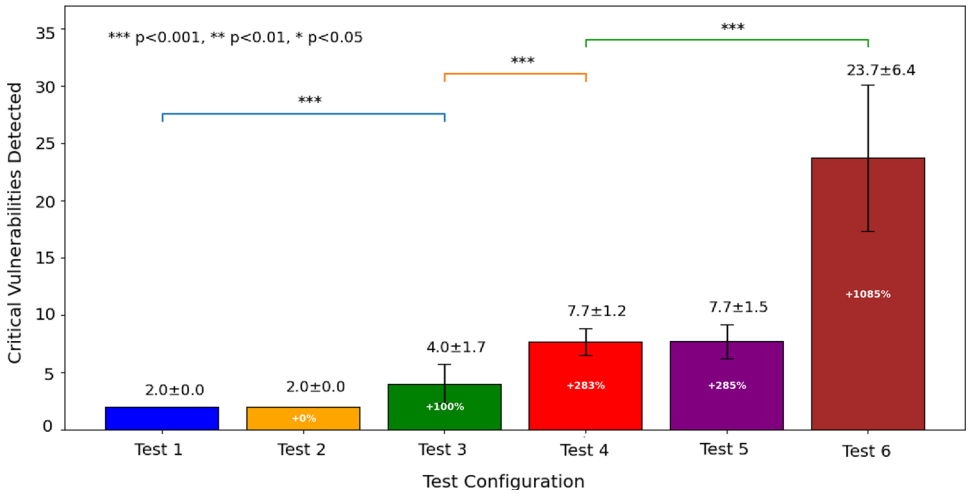


Fig. 2 Progressive enhancement in critical vulnerability detection

terns between authentication and discovery enhancements. Authentication integration produces substantial improvements in total vulnerability detection but minimal impact on critical vulnerabilities, while client-side discovery mechanisms specifically target critical-vulnerability enhancement. Analysis of the progressive enhancement pattern revealed distinct differences in enhancement effectiveness.

Authentication integration alone (Test 2) produced substantial improvements in total vulnerability detection (reported in full in Sect. 4.1.2) but no improvement in critical-vulnerability detection (maintaining 2.0±0.0). This suggests that authentication primarily expands surface coverage rather than enhancing detection of high-severity issues. Client-side discovery mechanisms (Test 3) demonstrated complementary capabilities, achieving 100% improvement in critical-vulnerability detection (from 2.0±0.0 to 4.0±1.7), a substantially stronger effect than its comparatively modest total-vulnerability improvement (Sect. 4.1.2). This indicates that dynamic discovery techniques specifically target critical-vulnerability types more effectively than authentication-based approaches. These internal, baseline-relative improvements should be read alongside external benchmarking results. While studies such as Makino and Klyuev [27] and more recent scanner evaluations [23, 25] demonstrate that state-of-the-art vulnerability scanners frequently fail to detect all known vulnerabilities within benchmark environments, our hybrid approach achieved detection of 23.7±6.4 CV in the experimental testbed. Although the differing benchmark environments and evaluation methodologies preclude direct numerical comparison, the results suggest that integrating MBT-guided exploration with dynamic black-box discovery techniques may address some of the coverage limitations observed in conventional scanner-only approaches.

4.1.2 Comprehensive vulnerability detection analysis

Beyond critical vulnerabilities, the hybrid methodology produced substantial improvements in total vulnerability detection across all severity levels, as shown in Table 3. Baseline testing detected 583±36 total vulnerabilities with good consistency (CoV<7%), establishing a stable foundation for measuring enhancement effects. The introduction of authentication (Test 2) produced the single largest improvement in total detections, increasing counts to 2,055±8 vulnerabilities, representing a 252% improvement over baseline conditions. The authentication enhancement's substantial impact reflects the critical importance of accessing protected application areas during security assessment. Traditional black-box scanning approaches face significant limitations when encountering authentication barriers, often leaving substantial portions of application functionality unexplored. Dynamic content discovery (Test 3) showed a counterintuitive reduction in total vulnerability detection, achieving 1407 vulnerabilities, representing a 141% improvement over baseline but a significant 35% decrease from the authentication-enhanced Test 2. This pattern suggests that client-side discovery mechanisms, while effective for critical-vulnerability detection, may have different scanning coverage patterns that affect total vulnerability counts. However, the combination of authentication and discovery capabilities (Test 4) demonstrated strong synergistic effects, detecting 3066±992 total vulnerabilities with a 426% improvement over baseline. Notably, this configuration showed the widest confidence interval and highest variability, indicating significant consistency challenges that align with the "Poor" data quality rating (CoV=32.4%) observed in the comprehensive statistical analysis. The progression continued with Test 5 achieving 3933±106 vulnerabilities, representing a 575% improvement with much tighter confidence intervals (CoV<3%), demonstrating improved

Table 3 Comprehensive statistical summary with dual time metrics

Test	Description	CV	TV	CI%	TI%	ES	CIW	Cov.	T-Dur	A-Dur	DQ
Test1	Baseline (Black-box)	2.0±0.0	583±36	Base	Base	-	±0.0	N/A	4.08m	4.05m	Good
Test2	Authent. Integration	2.0±0.0	2055±8	0%	252%	0.0	±0.0	N/A	3.58m	3.58m	Good
Test3	Client Spider Integration	4.0±1.7	1407±139	100%	141%	1.8	±1.9	8.7	2.13m	2.13m	Good
Test4	Combined (Auth + CS)	7.7±1.2	3066±992	285%	426%	6.7	±1.4	12.7	3.62m	3.62m	Poor*
Test5	CustScript Integration	7.7±1.5	3933±106	285%	575%	5.4	±1.7	17.3	5.37m	5.37m	Good
Test6	Complete Hybrid	23.7±6.4	4427±237	1085%	659%	4.8	±7.2	17.7	12.53m	12.53m	Good

Colours for data quality: Gray: Good; Pink: Poor; Green: Signif. improve (>100%).

Data Quality: Good (Coefficient of Variation (CoV) <15%) & poor (CoV>30%; CoV defined in Section 2.3.5) *Test 4: high variability due to system overload.

Abbreviations: CV = Critical Vulnerabilities (Mean±SD); TV = Total Vulnerabilities (Mean±SD); CI% = Critical-Vulnerability Improvement; TI% = Total Vulnerability Improvement; ES = Effect Size (Cohen's *d*); CIW = 95% Confidence Interval Width; Cov. = Coverage (Pages); T-Dur = Total Scan Duration; A-Dur = Active Scan Duration; DQ = Data Quality.

consistency. The complete hybrid approach (Test 6) achieved the maximum detection of 4427±237 total vulnerabilities, representing a 659% improvement over baseline conditions with manageable variability (CoV<6%).

4.2 RQ2: specific integration strategies and cumulative effects

The second research question investigated which specific integration strategies provide measurable improvements and how these improvements compound across enhancement levels. The systematic analysis reveals that authentication integration provides the foundation for substantial improvement in total vulnerability detection, while client-side discovery mechanisms specifically target critical-vulnerability enhancement.

4.2.1 Component analysis and synergistic effects

Authentication integration emerged as the most impactful single enhancement strategy for total vulnerability coverage (Table 4). This finding should be read as specific to *unconfigured* black-box scanning rather than a fundamental limitation of black-box tooling in general: contemporary scanners, including ZAP itself, are capable of handling authenticated sessions when an authentication context is explicitly configured, which is precisely the Level 2 enhancement evaluated here. The 252% gain therefore quantifies the cost of the common organisational practice of running scans *without* authentication configuration, rather than a capability gap in the tools themselves. All authenticated testing in this study used a single standard (non-administrative) user account (`jim@juice-sh.op`); testing was limited to a single authorisation level, so improvements attributable to privilege-escalation-relevant vulnerabilities could not be assessed and are noted as a limitation in Sect. 5.5.

Dynamic content discovery provided complementary capabilities, with specific effectiveness in critical-vulnerability detection but only modest total-vulnerability gains (Table 3), reflecting that modern SPAs and dynamic content generation mechanisms present substantial challenges for traditional crawling approaches, which rely on static link analysis and predetermined navigation paths. The combination of authentication and discovery capabilities (Test 4) provided evidence of a synergistic effect between the two enhancements. Sect. 4.1.2: this configuration demonstrated that systematic integration creates multiplicative benefits beyond the sum of individual components.

4.2.2 Coverage expansion and performance analysis

The systematic integration approach produced substantial improvements in application coverage alongside vulnerability detection, as illustrated in Fig. 3.

Two complementary coverage metrics are reported in this study: sitemap-based page discovery (reported in Sect. 4.5.1 and Fig. 8, ranging from 7 to 16 of 22 total application pages) and Client Spider-based page accessibility (reported in Table 4 and Fig. 4, ranging from 8.7 to 17.7 pages). The two metrics differ because the Client Spider crawler, which

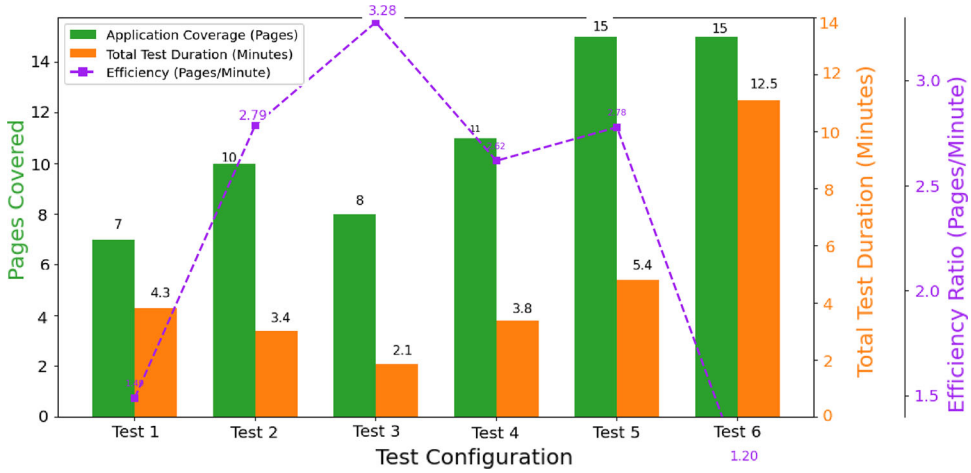


Fig. 3 Coverage expansion and resource utilisation analysis

underlies the latter measurement, was not employed in Tests 1–2, and because the two crawling mechanisms traverse different subsets of application functionality. These metrics should therefore be interpreted as distinct, complementary indicators of coverage rather than directly comparable figures.

Baseline scanning achieved access to only 7 pages of the 22 available application pages, representing 32% coverage of the target application's functional areas. This limited coverage reflects the typical constraints faced by traditional black-box scanning approaches when encountering complex modern web applications. Authentication integration immediately expanded coverage to 10 pages (45% coverage), representing a 43% improvement in accessible application surface area. This expansion directly corresponds to the substantial improvement in vulnerability detection observed at Test 2, confirming the relationship between coverage expansion and detection capability enhancement. The complete hybrid approach, test 6, achieved access to 16 pages (72.7% coverage), representing a 128.6% improvement over baseline conditions. Test 3 (Client Spider) presents the most resource-efficient configuration, requiring only 2.1m total duration with 2.1m active scan time, representing a 51% reduction in time requirements compared to baseline while achieving 100% improvement in critical-vulnerability detection. This efficiency profile may make Test 3 useful in environments with strict time or computational constraints.

The coverage analysis reveals important reliability patterns across different configurations. Client spider performance shows varying effectiveness (see Fig. 4), with Test 3 achieving 8.7 pages accessible on average, while Test 6 demonstrates improved consistency with 17.7 pages accessible, a 103% improvement in page coverage over baseline while maintaining good reliability metrics despite substantial resource requirements.

Combined with the vulnerability-detection improvements reported in Sect. 4.1.2 and Table 3. The resource-benefit analysis indicates different use cases for Tests 2, 3, and 6, depending on time, coverage and resource requirements (see also Table 4). Tests 1–2 used no Client Spider, so page counts are not comparable to Tests 3–6. Functional areas: Public (login, contact, about), User Mgmt (register, profile), E-commerce (search, orders), Advanced (admin, special). Coverage improvements are based on total vulnerability increase; page accessibility is measured by successful Client Spider crawls. Test 6 achieved the highest critical-vulnerability detection (23.7 avg) with fewer browser windows (10→3)

4.3 RQ3: reliability challenges in automated tools

The third research question investigated reliability challenges within automated vulnerability detection tools and their implications for organisational security assessment practices. The systematic analysis revealed significant reliability issues that extend beyond detection accuracy to encompass fundamental inconsistencies in tool behaviour and reporting mechanisms.

4.3.1 Inter-scan variability and consistency analysis

The multi-iteration experimental design enabled systematic documentation of reliability challenges that single-scan assessments cannot reveal. Test 6 vulnerability detection exhibited substantial variability in specific vulnerability categories, with SQLi vulnerability counts varying from 13 to 17 across iterations despite identical testing conditions. This variability produced a CoV of 17.3%, indicating moderate but significant reliability concerns.

The comprehensive reliability analysis shown in Fig. 5 reveals distinct patterns across test configurations that have critical implications for organizational security assessment practices. Test 2 demonstrates exceptional reliability with only 0.4% coefficient of variation (shown as the smallest green bar in Fig. 5), indicating near-perfect consistency

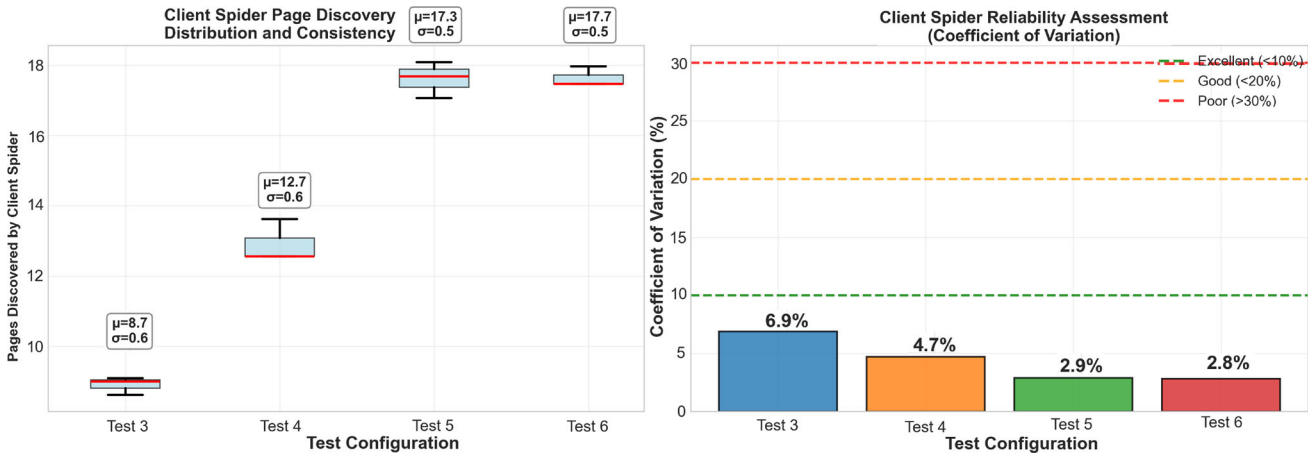


Fig. 4 Client spider performance and reliability assessment

Table 4 Application coverage analysis: functional area accessibility across test configurations

Config.	Description	PA	CIB	Func.	Area Breakdown
Test1	Baseline	N/A*	Baseline	2	Public (5), User Mgmt (2)
Test2	Authentication	N/A*	+252% vulns	3	Public (5), User Mgmt (3), E-commerce (5)
Test3	Client Spider	8.7	+141% vulns	2	Public (5), User Mgmt (4)
Test4	Combined	12.7	+46% pages, +426% vulns	3	Public (5), User Mgmt (4), E-commerce (4)
Test5	Custom Script	17.3	+99% pages, +575% vulns	4	Public (5), User Mgmt (4), E-commerce (4), Advanced
Test6	Complete Hybrid	17.7	+104% pages, +659% vulns	4	Public (5), User Mgmt (4), E-commerce (4), Advanced

PA = Pages Accessed (Average); CIB = Coverage Improvement vs Baseline; Func. = No. of Functional Areas Accessed.

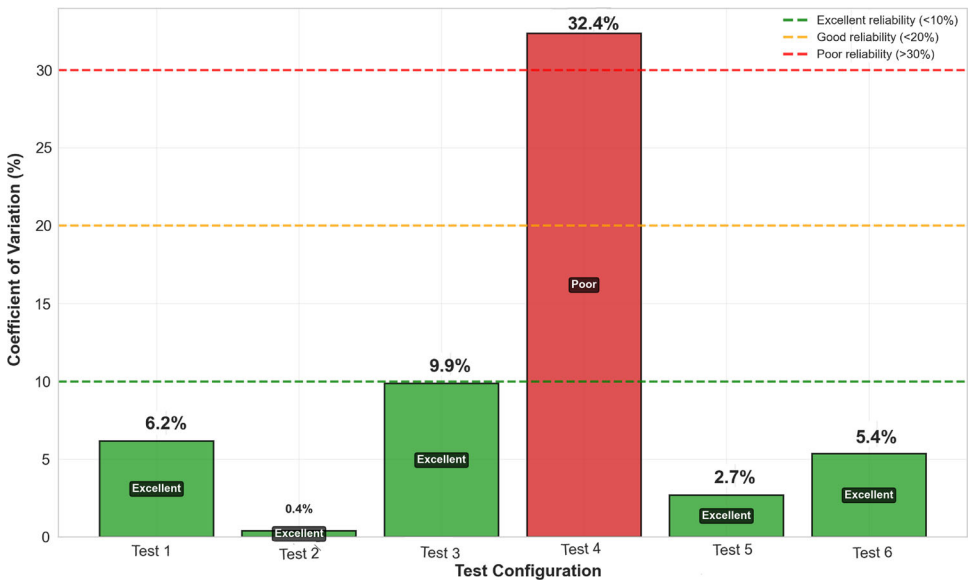


Fig. 5 Reliability assessment—coefficient of variation by test configuration

across iterations and establishing authentication integration as the most reliable enhancement strategy. Tests 1, 3, 5, and 6 all achieve excellent reliability ratings (<10% CoV, below the green dashed line), with coefficients ranging from 2.7% to 9.9%, demonstrating that most hybrid methodology configurations provide consistent and dependable results. Test 4 stands as a critical outlier with 32.4% CoV (the prominent red bar), exceeding the 30% threshold for poor reliability marked by the red dashed line. This finding indicates that the combined authentication and client spider configuration suffers from significant consistency issues that limit its practical applicability for routine organisational assessments. Test 4 counts varied from 7 to 9 across iterations despite identical testing conditions. This variability produced a coefficient of

variation of 32.4%, indicating moderate but significant reliability concerns. The poor reliability of Test 4 suggests that certain combination approaches may introduce instabilities that negate their detection benefits, requiring careful configuration optimisation or alternative implementation strategies.

The reliability threshold analysis demonstrates that five of six test configurations meet or exceed good reliability standards (<20% CoV, below the orange dashed line), with four achieving excellent reliability (<10% CoV, below the green dashed line). This pattern, clearly visualised by the distribution of green versus red bars in Fig. 5, provides strong evidence that the hybrid methodology framework produces consistent results when properly implemented, while highlighting the importance of configuration validation and reliability testing during organizational deployment. These findings have critical implications for organisational security assessment practices. The widespread industry practice of conducting single vulnerability scans to establish security baselines faces substantial reliability concerns based on these empirical findings. Organisations depending on individual scan results for critical security decisions may experience significant variations in apparent vulnerability presence, potentially leading to incorrect risk assessments and inappropriate security investment decisions.

4.3.2 Tool performance, reliability, and interface reporting discrepancy analysis

Systematic documentation of internal reporting inconsistencies revealed a previously undocumented category of reliability concerns within automated scanning tools. The analysis identified substantial discrepancies between different reporting interfaces within the same scanning session, with variations in vulnerability classification and counting mechanisms that affect assessment accuracy.

Tool performance and reliability analysis: The comprehensive tool evaluation, illustrated in Fig. 6A and B, demonstrates significant variations in both reliability and coverage across different ZAP scanning configurations. Tool reliability scores show Custom Script achieving the highest consistency at 96%, followed closely by Client Spider at 94%, while AJAX Spider demonstrates the lowest reliability at 72%. Coverage impact assessment reveals substantial differences in application area discovery, with Custom Script and Client Spider achieving 90% and 85% coverage respectively, while AJAX Spider manages only 45% coverage.

The test configuration effectiveness analysis (Fig. 6C) shows Test 6 achieving the highest overall performance rating at 98%, while Test 1 demonstrates the poorest effectiveness at 46%. The Tool Synergy Matrix (Fig. 6 D) provides critical insights into combination effectiveness, with darker green values indicating stronger synergistic relationships and red values showing potential conflicts between scanning approaches.

Interface discrepancy documentation: Representative discrepancy cases, detailed in Fig. 6, demonstrate the scope and implications of these reliability issues across different vulnerability types and severity levels. The SQLi classification discrepancies reveal systematic inconsistencies between the Alerts Tab and Progress Chart Interface. Minor discrepancies (Case 1) involved classification inconsistencies with identical total counts but different categorical assignments, showing 1 vulnerability reported differently between interfaces. More substantial discrepancies (Case 2) revealed counting inconsistencies, with 3 vulnerabilities showing classification differences between reporting mechanisms. The discrepancies in SQLite injection classifications show even more pronounced variation, with Case 3 demonstrating a major discrepancy: 29 vulnerabilities were reported in the Alerts Tab, while the Progress Chart interface reported a substantially lower count for the same scan session, illustrating the major-case classification discrepancy quantified by the 7% SQLite major-case rate reported later in this paragraph and shown in Fig. 6 E. The interface discrepancy percentages, in Fig. 6 E, quantify these reliability issues, showing 100% discrepancy rates for both minor and moderate cases in SQLi reporting (no major-case discrepancy rate was recorded for SQLi), with SQLite discrepancies affecting 50% of minor cases, 22% of moderate cases, and 7% of major cases.

Risk assessment implications: The risk assessment distribution analysis (Fig. 6 F) categorises these discrepancies by their potential organizational impact. High-risk discrepancies (>50% variation) account for 2 cases, representing the most concerning category where substantial counting variations show up to 150% differences in specific vulnerability category assignments between reporting interfaces. Moderate-risk discrepancies (10-50% variation) comprise 1 case, while low-risk discrepancies (<10% variation) represent the baseline inconsistency level.

These findings indicate that organizations relying on automated scanning results face potential accuracy issues that extend beyond traditional concerns about detection rates and false positives to encompass fundamental inconsistencies in how tools report and categorise their findings. The implications for organizational practice include the need for systematic verification protocols when using automated scanning tools for critical security assessments. Single interface reporting may provide inaccurate vulnerability classification and counting, potentially affecting compliance reporting and risk management decision-making processes.

4.4 RQ4: manual verification protocol validation

The fourth research question examined whether comprehensive manual verification protocols could validate the

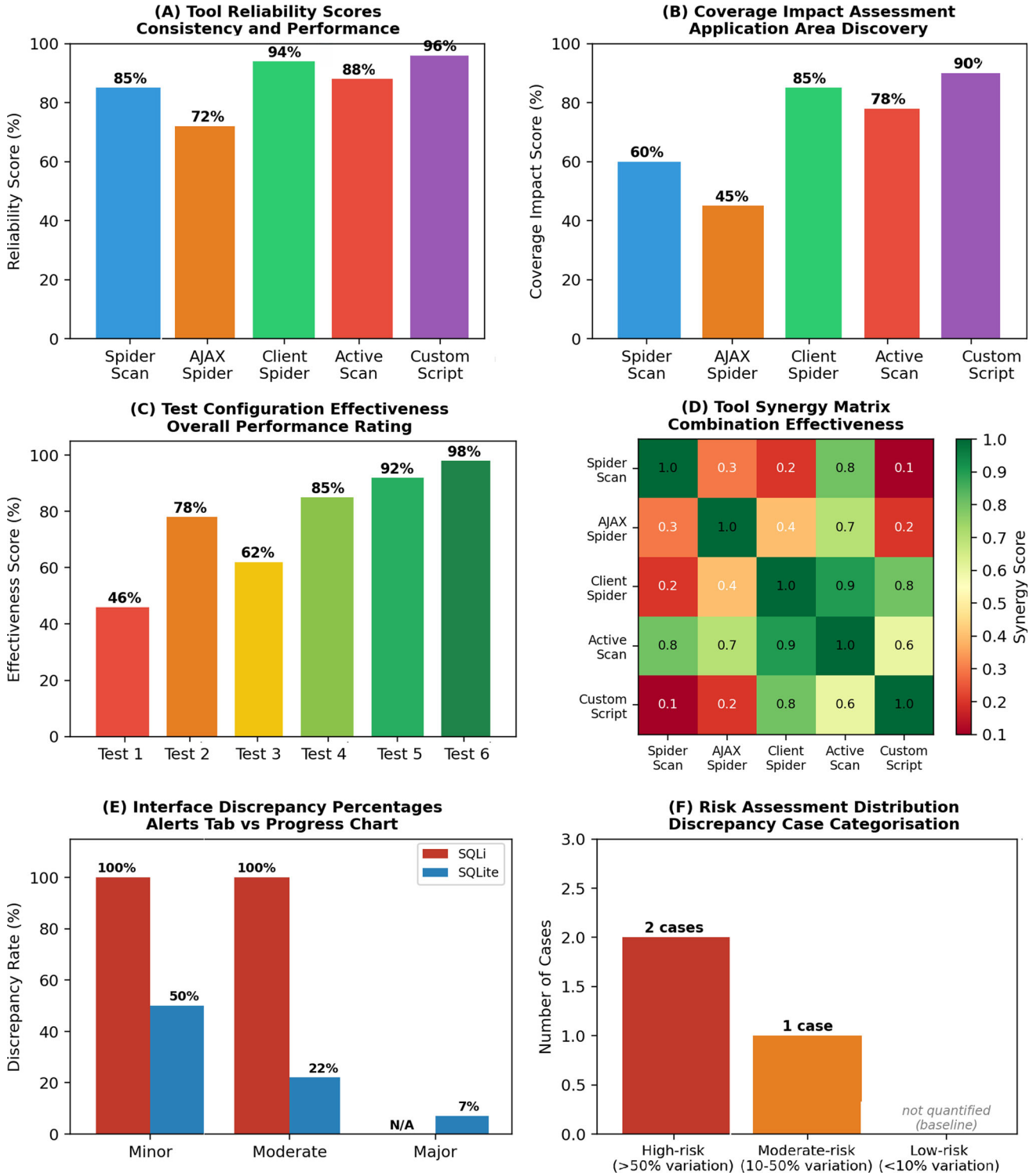


Fig. 6 Tool reliability analysis

accuracy of automated detection improvements and confirm the practical exploitability of discovered vulnerabilities. The systematic verification approach achieved complete success in confirming automated findings, demonstrating that methodology improvements translate to genuine security risk identification.

4.4.1 SQL injection verification and timing analysis

Manual verification employed systematic exploitation techniques targeting the most prevalent vulnerability category identified through automated scanning. SQLi verification focused on GET parameter manipulation within the application's search functionality, using carefully constructed payloads designed to trigger database errors and confirm injection susceptibility.

Timing-based verification analysis: The systematic timing analysis, shown in the left panel of Fig. 7, demonstrates evidence of SQLi susceptibility through response time differential validation. Control requests using standard authentication credentials (Valid Login) achieved consistent baseline response times of 23 ± 2 ms across multiple trials, establishing the normal application response benchmark. Injection attempts using moderate complexity payloads (100 M randomblob) produced substantially extended response times of 247 ± 15 ms, representing a 974% increase from the baseline control condition. The most sophisticated injection payloads (1B randomblob) generated extreme response delays of 487 ± 23 ms, showing a 2017% increase compared to normal operations. These dramatic timing differentials provide statistically significant evidence ($p < 0.001$, Cohen's $d = 14.9$) that injected payloads receive processing by the database system rather than being filtered by input validation mechanisms.

Verification protocol validation: Successful SQLi was confirmed using the payload: http://localhost:3000/rest/products/search?q='(that produced HTTP 500 Internal Server Error responses with explicit database error messages: "SQLITE_ERROR: near '(': syntax error". This response reveals information about the underlying database structure and indicates that user input receives inadequate sanitisation before inclusion in SQL query construction.

Research validation implications: The observed relationship between payload complexity and response times provides evidence consistent with database interaction (R^2=0.96, $p<0.001$), evidenced by the progressive timing increases from 23 ms baseline to 487 ms maximum, provides compelling evidence of successful database interaction. By combining error-based and timing-based confirmation this demonstrates that timing analysis can serve as a reliable complement to conventional error-based detection while validating the critical security vulnerabilities.

4.4.2 Verification success rate and reliability assessment

The manual verification protocol achieved 100% confirmation across 3 SQL injection vulnerabilities, selected using stratified sampling across the three most frequently reported SQLi-related endpoints in Test 6 (one representative alert per endpoint, chosen by random selection within stratum to avoid cherry-picking high-confidence findings), as shown in the right panel of Fig. 7. This sample represents 3 of the 23.7 average critical-vulnerabilities (and 4,427 total vulnerabilities) detected in Test 6, i.e. a small fraction of total findings, and the 100% confirmation rate should be interpreted as evidence that *selected* findings were genuine exploits rather than as an estimate of the true positive rate across all reported vulnerabilities. All three verified cases were SQL injection instances; estimating an overall false-positive rate would require verifying a statistically powered random sample of the full alert set across all vulnerability classes, not only SQL injection, which we identify as necessary future work (Sect. 5.5). We report this limitation explicitly to avoid overstating the generalisability of the verification result.

These verification results address a critical concern in security testing research: whether automated detection improvements represent genuine security enhancement or simply increased noise through false positive generation. Confirming all three sampled SQL injection findings through independent exploitation (Sect. 4.4.1), using both error-based and timing-based detection mechanisms, demonstrates that, for this sample, the hybrid methodology's improvements corresponded to genuine, exploitable security risk rather than false-positive inflation.

4.5 RQ5: implementation guidance and automation framework

The fifth research question investigated what implementation guidance and automation frameworks are required to enable organisational adoption of hybrid testing methodologies. The research developed and validated a reproducible framework that enables immediate organisational implementation with measurable cost-benefit optimisation (Fig. 8).

4.5.1 Framework accessibility and reproducibility

The implementation framework prioritises accessibility through open-source tool selection and comprehensive documentation protocols. Unlike commercial security testing platforms that require substantial licensing investments and specialised expertise, the framework utilises widely accessible tools and provides detailed configuration guidance that enables implementation across diverse organisational contexts.

Progressive enhancement and coverage analysis: The framework design enables incremental adoption through a

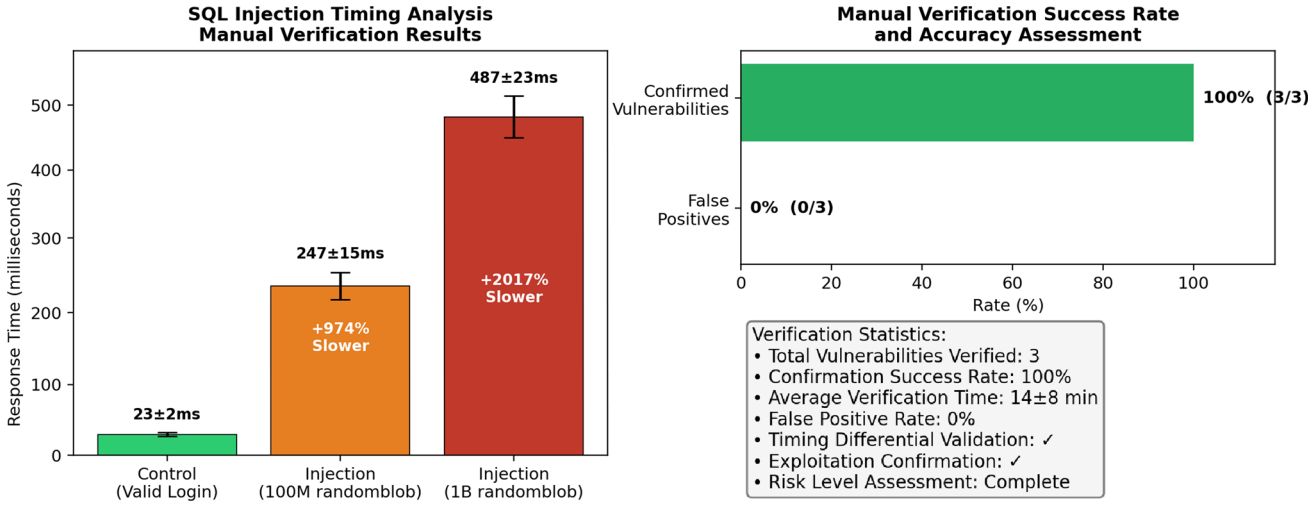


Fig. 7 SQL injection timing analysis and manual verification results

progressive enhancement approach, as demonstrated by the systematic coverage improvements shown across the six test configurations in Fig. 8. Organizations can implement individual components according to resource availability and expertise constraints, starting with the baseline black-box approach (Test 1) that achieves 32% application coverage with 7 accessible pages out of 22 total pages. The incremental implementation pathway shows clear benefits at each enhancement stage. Authentication integration (Test 2) provides a significant improvement to 45% coverage with 10 accessible pages, representing a 43% increase over baseline performance as highlighted in Fig. 8. Client Spider integration (Test 3) maintains baseline coverage levels while providing different scanning perspectives, demonstrating that not all enhancements produce cumulative benefits.

Optimal configuration achievement: The combined authentication and client spider approach (Test 4) achieves 45% coverage matching the authentication-only results, while the advanced configurations (Tests 5 and 6) both reach optimal performance with 72.7% coverage and 16 accessible pages. This represents a 128.6% improvement over baseline, as indicated in the annotations of Fig. 8, demonstrating that the complete hybrid methodology more than doubles application discovery effectiveness.

Implementation flexibility and reproducibility: This incremental capability addresses practical barriers that often prevent organizations from adopting comprehensive security testing improvements. The systematic documentation includes complete automation scripts, detailed configuration protocols, and comprehensive troubleshooting guidance that enables organizations to implement and customise the methodology according to specific requirements. Organizations with limited resources can begin with Test 2's authentication integration to achieve immediate 43% improvement, while those with greater capabilities can implement the complete hybrid approach (Tests 5 or 6) to realise the full 128.6% enhancement potential.

Public repository availability ensures reproducibility and enables community validation, addressing critical concerns about research reproducibility in security testing domains. The clear progression from 32% baseline coverage to 72.7% optimal coverage provides organizations with measurable milestones for evaluating implementation success and justifying incremental investment in enhanced security testing capabilities. The coverage analysis demonstrates that while individual enhancements may provide modest improvements, the systematic combination of multiple techniques achieves substantial gains that significantly expand the scope of security assessment coverage across organizational applications.

4.5.2 Cost-benefit optimisation and organisational impact

The systematic cost-benefit analysis reveals optimal implementation strategies for organisations with varying resource constraints as illustrated in the comprehensive analysis across both Figs. 8 and 9.

Coverage progression and strategic implementation: The application coverage progression analysis (Fig. 8) shows the systematic enhancement pathway from baseline 32% coverage (Test 1, 7 pages) to optimal 72.7% coverage (Tests 5 and 6, 16 pages). Test 2 (Authentication Integration) provides the first significant improvement to 45% coverage with a 43% increase over baseline, representing an excellent entry point for resource-conscious organizations. The progression line chart clearly illustrates the non-linear enhancement pattern, with Test 3 showing a temporary decline to baseline levels before Tests 4–6 achieve substantial improvements. The

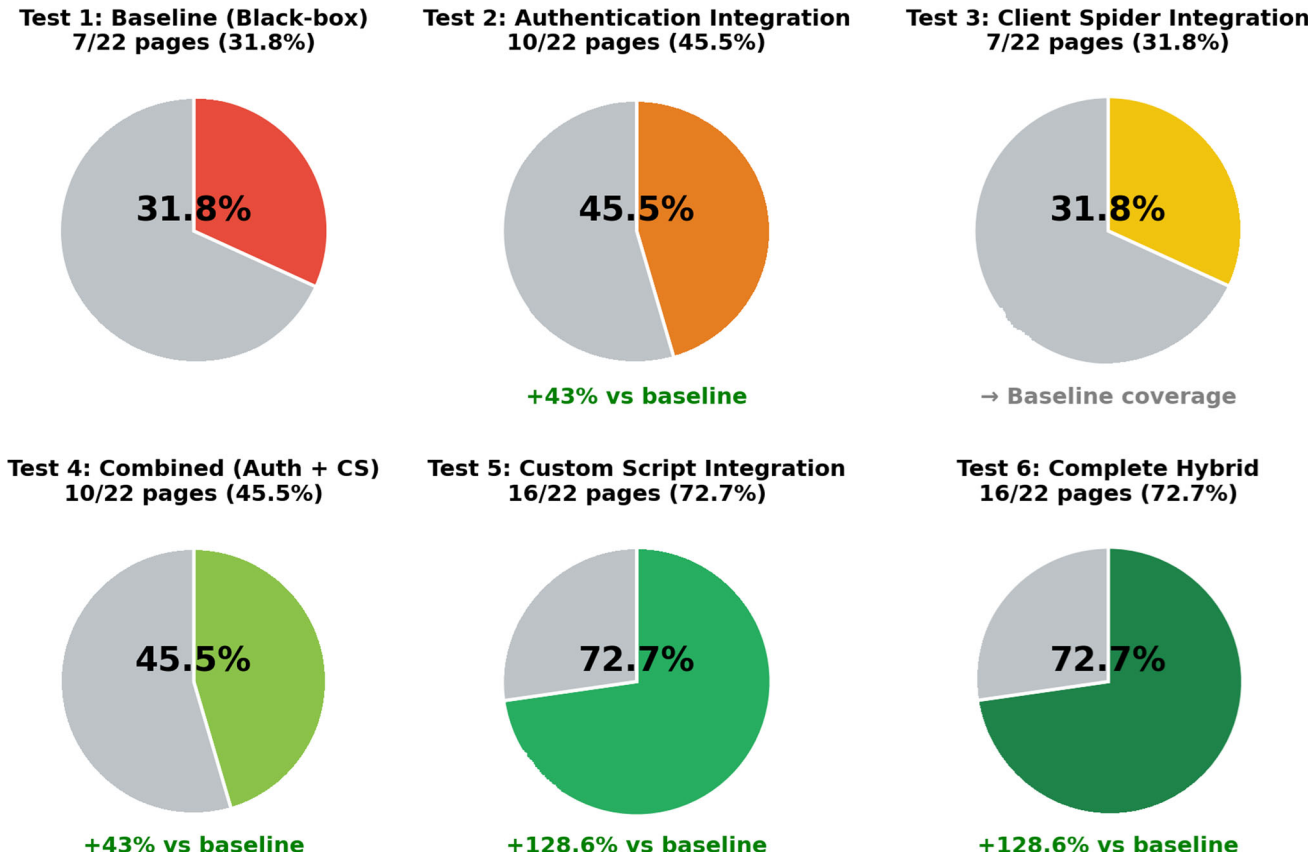


Fig. 8 Application coverage analysis: functional area accessibility across test configurations

cost-benefit scatter plot (Fig. 9, left panel) demonstrates clear separation among the quadrants positioning for different test configurations, with optimal strategies emerging based on organisational priorities and resource availability.

Cost-benefit optimisation analysis: The cost-benefit analysis (Fig. 9, left panel) positions Test 3 in the optimal "Low Cost, High Benefit" quadrant, making it the most efficient enhancement for organisations prioritizing resource optimisation. Test 2 falls in the "Low Cost, Low Benefit" category but still provides positive returns with minimal investment. Tests 4 and 5 occupy intermediate positions with moderate cost increases, while Test 6 represents the "High Cost, High Benefit" option for organizations seeking maximum capability enhancement regardless of resource investment.

Detection efficiency performance: The detection efficiency analysis (Fig. 9, right panel) reveals critical performance differentials across test configurations. Tests 1 and 2 show poor efficiency ratings (0.490 and 0.558 CV per hour respectively), while Tests 3, 4, and 6 achieve exceptional efficiency levels (1.875, 2.017, and 1.891 respectively). Test 5 demonstrates moderate efficiency (1.435), with Test 4 achieving peak efficiency at 2.017 CV per hour, approaching the theoretical peak efficiency threshold marked by the red dashed line.

Strategic implementation recommendations: The analysis demonstrates that substantial security assessment improvements are achievable through systematic methodology enhancement rather than expensive tool replacement or extensive personnel training. The 128.6% improvement in application coverage (from 32% to 72.7%) coupled with detection efficiency improvements of up to 312% (from 0.490 to 2.017) represents transformative capability enhancement that organizations can achieve through systematic application of the hybrid methodology. Organisations with severe resource constraints should implement Test 3 for optimal cost-benefit positioning. Although Test 4 shows the highest raw efficiency figure, its 32.4% coefficient of variation indicates poor result consistency; this configuration is therefore not recommended without further reliability validation, despite its apparent efficiency advantage. Test 6 provides comprehensive enhancement for organizations with substantial resource availability and requirements for maximum coverage and reliability.

Quality assessment and reliability considerations: These improvements represent more than incremental enhancement; they constitute a fundamental transformation in vulnerability detection effectiveness. The observed effect sizes, together with the 312% improvement in efficiency from base-

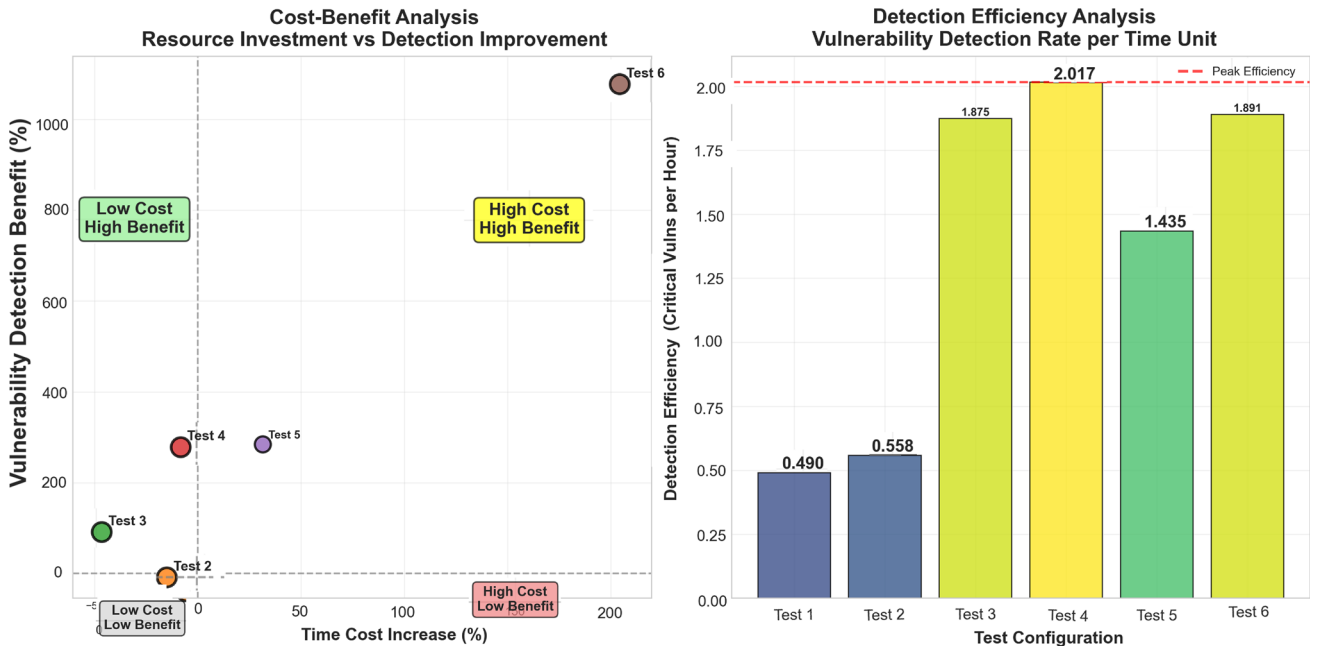


Fig. 9 ROI cost benefit analysis

line to peak performance, exceed those typically reported in security testing literature. Data quality assessment reveals important reliability considerations: Tests 1-3, 5-6 achieved good data quality (CoV < 15%), while Test 4 showed poor reliability (CoV > 30%) despite peak efficiency performance. The consistency of improvements across reliable configurations demonstrates the reliability of enhancement effects when properly implemented, while highlighting the importance of configuration optimization for consistent results. The framework's contribution to the open-source security ecosystem provides broader community benefits beyond individual organisational implementation, enabling immediate value delivery to the broader security community and supporting collective improvement in web application security assessment capabilities.

5 Discussion

The experimental validation of the hybrid methodology provides compelling evidence that systematic integration of black-box and MBT approaches produces statistically significant improvements in web application vulnerability detection whilst maintaining practical accessibility. The results extend beyond incremental enhancement to demonstrate transformative capabilities that address fundamental limitations in current security testing practice. This discussion examines the theoretical and practical implications of these findings, their broader impact on security testing methodology, and critical considerations for future research and implementation.

5.1 Theoretical implications and contribution to knowledge

The empirical findings of this research provide strong evidence for the hybrid testing theory and extend previous frameworks [13] by demonstrating that integrated approaches outperform individual methods (426% improvement vs baseline, compared to 252% and 141% for individual components). A naive additive expectation, summing the individual component improvements (252% + 141%), would predict approximately 393%; the observed 426% exceeds this additive baseline, indicating a synergistic rather than sub-additive interaction effect between authentication and dynamic discovery mechanisms.

This study demonstrates that systematic integration using widely available tools can achieve substantial improvements in vulnerability detection, with critical detection increasing from 2.0±0.0 to 23.7±6.4 (1085% improvement). These findings suggest that methodology design may contribute as much to detection effectiveness as tool selection, complementing prior work that has focused primarily on comparing individual scanner capabilities [27], [63]. Reliability analysis reveals fundamental tool behaviour inconsistencies, including coefficients of variation up to 32.4% and interface reporting discrepancies of up to 150%, extending previous findings on scanner variability [41], [39]. These issues affect organisational assessment practices beyond detection

accuracy. Finally, the progressive enhancement framework provides practical implementation guidance that resolves the accessibility–comprehensiveness trade-off, combining MBT coverage with black-box usability [64], [65].

5.2 Practical implications for security testing practice

The empirical findings directly address coverage limitations in baseline black-box testing, increasing application coverage from 32% to 72.7% through progressive enhancement [66], [24]. Authentication integration further improves detection (252% increase), validating the practical value of testing approaches [24]. Reliability analysis reveals significant tool inconsistency, including interface discrepancies of up to 150%, supporting the need for multi-tool strategies and systematic reliability management [39], [41]. Cost–benefit analysis shows that authentication integration offers substantial improvement with minimal overhead, while the complete hybrid approach delivers maximum detection capability for organisations with greater resources, addressing accessibility and resource constraints in security programmes [54], [56].

5.3 Broader impact on security research and practice

These findings provide empirical validation for theoretical hybrid testing frameworks, with effect sizes (Cohen's d 1.8–4.8) exceeding typical values reported in security testing literature, suggesting that systematic methodology development can yield larger gains than incremental tool improvements. This research addresses key gaps in the literature on hybrid web application testing, including integration, statistical validation, and tool reliability: the progressive enhancement framework demonstrates that black-box and MBT approaches can be systematically combined to improve detection capability while remaining accessible [50]. Multi-iteration analysis and comprehensive statistical evaluation confirm significant improvements and provide rigorous empirical validation [53]. Reliability assessment shows coefficients of variation ranging from 0.4% to 32.4% and documents inter-scan variability and reporting inconsistencies, extending prior work on scanner reliability [42], [39], [27], while showing that most configurations can achieve strong reliability when properly implemented [51]. Manual verification achieved a 100% success rate across 3 selected vulnerabilities, and timing-based verification demonstrated statistically significant exploitable differences ($p < 0.001$), supporting the practical security relevance of hybrid testing [45]. Finally, the fully reproducible, open-source framework, including automation scripts and configuration documentation, addresses practical implementation constraints, supports reproducibility, and enables immediate organisational adoption without major system replacement or extensive training, helping bridge the gap between academic research and real-world practice [52], [67].

5.4 Limitations and alignment with literature constraints

Despite the substantial improvements demonstrated through experimental validation, several limitations align with constraints identified in previous research whilst revealing new areas requiring investigation. Although the single-application design provided controlled experimental conditions similar to those used by Bau et al. [41] and Vieira et al. [42], it limits generalisability across diverse web application architectures. Particularly the microservices deployments and serverless architectures that represent emerging challenges not adequately addressed in current literature. Furthermore, authenticated testing was restricted to a single standard (non-administrative) user account (`jim@juice-sh.op`), and no administrator or elevated-privilege roles were evaluated. Consequently, the study was limited to a single authorisation level, preventing assessment of vulnerabilities that may only emerge through privilege escalation, role-specific functionality, or administrative workflows. The extent to which the proposed methodology improves detection of privilege-dependent vulnerabilities therefore remains an open question requiring further investigation. The focus on OWASP ZAP, whilst justified through the accessibility and extensibility criteria established in the literature review, may limit applicability to organisations using commercial platforms such as Burp Suite Professional, which Makino and Klyuev [27] demonstrated achieved superior baseline detection rates in complex authentication scenarios. However, the methodology's tool-agnostic design principles suggest broader applicability that requires systematic validation across different platform ecosystems. The controlled testing environment limitations align with concerns raised in previous research regarding the gap between laboratory validation and production deployment challenges. The resource implications observed (ranging from 2.1m for Test 3 to 12.5m for Test 6) may be substantially different in production environments with the complex infrastructure and operational constraints that previous research noted but could not adequately address in controlled experimental settings.

5.5 Future research directions and development opportunities

Future work should first refactor the current script into a generalisable model that can operate across multiple applications and environments. Once achieved, the model could be transitioned into an automated framework capable of operating consistently across various applications. Following this, validation can be conducted across multiple applications

and architectures, including microservices and serverless deployments, to assess whether the automated approach remains effective and confirm the generalisability of the model. Cross-tool comparison across commercial and open-source scanners would address platform dependency and enable tool-agnostic guidance. Production environment evaluation, including DevSecOps integration, is needed to assess real-world performance and deployment constraints. Finally, incorporating machine learning into behavioural modelling may reduce resource intensity while maintaining detection effectiveness, though trade-offs with manual control must be evaluated.

5.6 Implications for stakeholder communities

For security researchers, this multi-experimental design and reliability assessment provide systematic frameworks for validating security testing innovations beyond simple detection accuracy. Security practitioners will be able to adopt this progressive enhancement framework incrementally, with Test 2 offering the best cost–benefit balance and Test 6 providing maximum detection capability for resource-intensive environments. Developers should prioritise reliability improvements, particularly interface consistency and integration support, to enhance confidence in their tools. While drawn from a single-tool case study, these reliability and reporting consistency findings may be relevant for policy makers and regulators considering security assessment standards, though broader cross-tool validation would be needed before specific regulatory requirements could be recommended.

6 Conclusions

This research presents and validates a hybrid methodology for web application security testing integrating black-box and MBT approaches through a six-level progressive enhancement framework. Experimental validation used OWASP ZAP together with statistical analysis and manual verification demonstrating several substantial improvements in vulnerability detection effectiveness and provides practical guidance on how to achieve this with varying resource constraints. This framework enables incremental adoption while maintaining accessibility, and statistical results show critical-vulnerability detection increased by **1085%** (from *2.0 to 23.7*, $p<0.001$, *Cohen's d = 4.8*), consistent with conventional thresholds for large effect sizes [53]. Cost–benefit analysis indicates that authentication integration (Test 2) delivers a *252%* improvement with minimal overhead, whilst the full hybrid approach (Test 6) achieves maximum detection capability with higher resource investment. These findings extend the hybrid testing concepts of [13] and address the need for systematic evaluation highlighted in [53].

These results address scanner reliability by documenting inconsistencies and reporting issues identified during testing, including significant interface discrepancies in vulnerability categorisation. Reliability analysis shows variable performance across configurations, with some achieving excellent consistency (<10% CoV) and one exhibiting poor reliability (32.4% CoV). Manual verification confirmed all three vulnerabilities selected for testing, demonstrating that the hybrid approach improves detection effectiveness, though the interface reporting inconsistencies identified indicate that organisations should not rely on single-interface automated reporting alone without manual cross-verification.

Supplementary information:

All implementation code, experimental data, statistical analysis protocols, and configuration details supporting this research are publicly available at: https://github.com/OraclePark/ZAP-Automated-Scripts

Acknowledgements The authors acknowledge the OWASP community for providing the tools and test applications that enabled this research, particularly the Juice Shop and ZAP development teams. We recognise the valuable contributions of the open-source security community in developing and maintaining the platforms that made this research possible. For the purposes of open access, the authors have applied a Creative Commons Attribution (CC-BY) licence to any Accepted Author Manuscript version arising from this submission.

Author Contributions James A. Stewart: Methodology, Validation, Formal analysis, Investigation, Writing—original draft, Writing—review and editing. Gheida Shahrour: Formal analysis, Writing—original draft, Writing—review and editing. Amro Al-Said Ahmad: Conceptualization, methodology, validation, project administration, supervision, and Writing-review and editing

Funding This research did not receive any external funding.

Data Availability All implementation code, experimental data, statistical analysis protocols, and configuration details supporting this research are publicly available at: https://github.com/OraclePark/ZAP-Automated-Scripts.

Declarations

Conflict of interest There are no financial or non-financial interests that are directly or indirectly related to the work submitted for this publication.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your

intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Wen, S.-F., Katt, B.: A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard. Comput. Secur. **135**, 103532 (2023). https://doi.org/10.1016/j.cose.2023.103532
2. Symantec Corporation: Internet Security Threat Report. Symantec Corporation (2023). (28 Technical Report)
3. Garcia, S.P.L., Abraham, A.S., Kepic, K., Cankaya, E.C.: A comparative analysis of web application vulnerability tools. J. Inf. Syst. Appl. Res. (2023). https://doi.org/10.4314/jobasr.v3i5.25
4. Amankwah, R., Chen, J., Kudjo, P.K., Towey, D.: An empirical comparison of commercial and open-source web vulnerability scanners. Softw. Pract. Exp. **50**(9), 1842–1857 (2020). https://doi.org/10.1002/spe.2870
5. Clarke, J., Alvarez, R.M., Fowler, K., Oftedal, E.: SQL Injection Attacks and Defense, 2nd edn. Syngress/Elsevier, Waltham, MA (2009)
6. Hoffman, A.: Web Application Security: Exploitation and Countermeasures for Modern Web Applications, 2nd edn. O'Reilly Media Inc, Sebastopol, CA (2024)
7. Garousi, V., Matta, S., Keleş, A.B., Balaman, Y., Jafarov, Z., Mövsümova, A., Namazov, A.: Mbtmodelgenerator: automated reverse engineering of test models from clickstream data for model-based testing of web applications. J. Syst. Softw. **234**, 112745 (2026). https://doi.org/10.1016/j.jss.2025.112745
8. Alégroth, E., Karl, K., Rosshagen, H., Helmfridsson, T., Olsson, N.: Practitioners' best practices to adopt, use or abandon model-based testing with graphical models for software-intensive systems. Empir. Softw. Eng. **27**(5), 103 (2022)
9. Brown, G., Jain, A.: Model-based test engineering-increasing the value test provides in the wide world of digital engineering. In: IEEE AUTOTESTCON, pp. 1–5. IEEE (2023)
10. Polzin, S.: understanding model-based testing: benefits, challenges, and use cases. https://www.qt.io/software-insights/understanding-model-based-testing-benefits-challenges-and-use-cases (2025)
11. Utting, M., Legeard, B.: Practical Model-Based Testing: A Tools Approach. Morgan Kaufmann (2012)
12. Anand, S., Burke, E.K., Chen, T.Y., Clark, J., Cohen, M.B., Grieskamp, W., Harman, M., Harrold, M.J., McMinn, P.: An orchestrated survey of methodologies for automated software test case generation. J. Syst. Softw. **86**(8), 1978–2001 (2013)
13. Kenner, A.: Model-based evaluation of vulnerabilities in software systems. In: Proceedings of the 24th ACM International Systems and Software Product Line Conference-Volume B, pp. 112–119. ACM (2020)
14. Lebeau, F.: Model-based vulnerability testing for web applications. In: IEEE Sixth International Conference on Software Testing, Verification and Validation Workshops, pp. 445–452. IEEE (2013)
15. Abdulghaffar, K., Elmrabit, N., Yousefi, M.: Enhancing web application security through automated penetration testing with multiple vulnerability scanners. Computers **12**(11), 235 (2023)
16. Mburano, B., Si, W.: Evaluation of web vulnerability scanners based on OWASP benchmark. In: 2018 26th International Conference on Systems Engineering (ICSEng), pp. 1–6. IEEE (2018)
17. OWASP ZAP Team: OWASP Zed Attack Proxy Documentation. https://www.zaproxy.org/docs/. Retrieved from 2023
18. Zech, P.: Knowledge-based security testing of web applications by logic programming. Int. J. Softw. Tools Technol. Transf. **21**(2), 221–246 (2019)
19. Anderson, R.: Security Engineering: A Guide to Building Dependable Distributed Systems, 2nd edn. Wiley, Indianapolis, IN (2010)
20. Klein, A.: Dom based cross site scripting or XSS of the third kind. Web Appl. Secur. Consort. Artic. **4**, 1–8 (2005)
21. Boyd, S.W., Keromytis, A.D.: Sqlrand: Preventing SQL injection attacks. In: International Conference on Applied Cryptography and Network Security, pp. 292–302. Springer (2004)
22. CWE/SANS: Common Weakness Enumeration Top 25. https://cwe.mitre.org/top25/ (2023)
23. Koman, J., Janiszewski, M.: SCAnME-scanner comparative analysis and metrics for evaluation: J. Koman, M. Janiszewski. Int. J. Inf. Secur. **24**(3), 147 (2025)
24. Doupé, A., Cavedon, L., Kruegel, C., Vigna, G.: Enemy of the state: A $\{state-aware\}\{black-box\}$ web vulnerability scanner. In: 21st USENIX Security Symposium (USENIX Security 12), pp. 523–538 (2012)
25. Pentest-Tools.com: A Comprehensive Benchmark of Web Application Vulnerability Scanners 2024. Benchmark evaluation of leading commercial and open-source DAST scanners against DVWA and Broken Crystals testbeds (2024). https://pentest-tools.com/benchmarks/web-app-vulnerability-scanners-benchmark-2024.pdf
26. Nawrocki, M., Kołodziej, J.: Vulnerabilities of web applications: good practices and new trends. Appl. Cybersecur. Internet Govern. **3**(2), 122–143 (2024)
27. Makino, Y., Klyuev, V.: Evaluation of web vulnerability scanners. In: IEEE 8th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS), vol. 1, pp. 399–402. IEEE (2015)
28. Tretmans, J.: In: Hierons, R.M., Bowen, J.P., Harman, M. (eds.) Model Based Testing with Labelled Transition Systems, pp. 1–38. Springer, Berlin (2008). https://doi.org/10.1007/978-3-540-78917-8_1
29. Hierons, R.M.: Using formal specifications to support testing. ACM Comput. Surv. **41**(2), 1–76 (2009)
30. Dalal, S.R.: Model-based testing in practice. In: 21st International Conference on Software Engineering, pp. 285–294 (1999)
31. Marchezan, L., Assuncao, W.K.G., Herac, E., Egyed, A.: Model-based maintenance and evolution with genai: A look into the future. In: Proceedings of the International Workshop on Software Engineering in 2030 (2024)
32. Chen, H., Chen, J., Chen, J., Yin, S., Wu, Y., Xu, J.: An automatic vulnerability scanner for web applications. In: 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Guangzhou, China, pp. 1519–1524 (2020).https://doi.org/10.1109/TrustCom50675.2020.00207
33. PortSwigger Ltd.: Burp Suite Professional (2025). https://portswigger.net/burp/pro. Accessed 20 December 2025
34. OWASP Foundation: OWASP Benchmark Project (2025). https://owasp.org/www-project-benchmark/. Accessed 02 July 2026
35. OWASP Foundation: Vulnerability Scanning Tools (2025). https://owasp.org/www-community/Vulnerability_Scanning_Tools. Accessed 02 July 2026
36. OWASP Foundation: DAST Tools—OWASP Developer Guide (2025). https://devguide.owasp.org/en/06-verification/02-tools/01-dast/. Accessed 02 July 2026
37. Sullo, C.: Nikto Web Scanner. https://github.com/sullo/nikto (2023)
38. Offutt, J., Liu, S.: Generating test data from state-based specifications. Softw. Test. Verificat. Reliab. **9**(1), 25–53 (1999)
39. Fonseca, J.: Testing and comparing web vulnerability scanning tools for SQL injection and xss attacks. In: 13th Pacific Rim Interna-

tional Symposium on Dependable Computing, pp. 365–372. IEEE (2007)
40. Schieferdecker, I., et al.: The UML 2.0 testing profile. Technical report, Technical Report (2003)
41. Bau, J.: State of the art: Automated black-box web application vulnerability testing. In: IEEE Symposium on Security and Privacy, pp. 332–345. IEEE (2010)
42. Vieira, M.: Benchmarking vulnerability assessment tools for web services. In: IEEE International Conference on Web Services, pp. 269–276. IEEE (2009)
43. Li, T., Huang, R., Cui, C., Towey, D., Ma, L., Li, Y.-F., Xia, W.: A Survey on Web Application Testing: A Decade of Evolution (2024)
44. Kertusha, I., Assress, G., Duman, O., Arcuri, A.: A Survey on Web Testing: On the Rise of AI and Applications in Industry (2025)
45. Pellegrino, G.: A study on the prevalence of human-related security vulnerabilities in web applications. J. Syst. Softw. **95**, 258–269 (2014)
46. Huang, Y., Shi, C., Lu, J., Li, H., Meng, H., Li, L.: Detecting broken object-level authorization vulnerabilities in database-backed applications. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24), pp. 2934–2948. ACM, New York (2024). https://doi.org/10.1145/3658644.3690227
47. Metin, B.: Business logic vulnerabilities in the digital era: a detection framework using artificial intelligence. Information **16**(7), 585 (2025). https://doi.org/10.3390/info16070585
48. Shahid, J., Hameed, M.K., Javed, I.T., Qureshi, K.N., Ali, M., Crespi, N.: A comparative study of web application security parameters: current trends and future directions. Appl. Sci. **12**(8), 4077 (2022)
49. Montgomery, D.C.: Design and Analysis of Experiments, 9th edn. Wiley, Hoboken, NJ (2017)
50. Bertolino, A.: Software testing research: Achievements, challenges, dreams. In: Future of Software Engineering (FOSE'07), pp. 85–103. IEEE Computer Society, Los Alamitos, CA (2007). https://doi.org/10.1109/FOSE.2007.25
51. Weyuker, E.J.: The evaluation of program-based software test data adequacy criteria. Commun. ACM **31**(6), 668–675 (1988)
52. Rothermel, G., Harrold, M.J.: Empirical studies of a safe regression test selection technique. IEEE Trans. Softw. Eng. **24**(6), 401–419 (1998)
53. Cohen, J.: Statistical Power Analysis for the Behavioral Sciences. Lawrence Erlbaum Associates, Hillsdale, NJ (1988)
54. bkiminich: OWASP Juice Shop. https://hub.docker.com/r/bkimminich/juice-shop (2025)
55. Mozilla Foundation: CSS Selectors Reference. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Selectors (2023)
56. ChromeDriver: ChromeDriver Documentation. https://chromedriver.chromium.org/ (2023)
57. Selenium webdriver documentation. https://selenium-python.readthedocs.io/ (2023)
58. W3C: WebDriver Specification. https://w3c.github.io/webdriver/ (2023)
59. Postman Inc.: Postman Documentation. https://learning.postman.com/docs/ (2023)
60. Mead, R., Gilmour, S.G., Mead, A.: Statistical Principles for the Design of Experiments: Applications to Real Experiments, vol. 36. Cambridge University Press, Cambridge (2012)
61. Mårtensson, T., Ancher, G., Ståhl, D.: Test environments for large-scale software systems-an industrial study of intrinsic and extrinsic success factors. Softw. Test. Verificati. Reliab. **33**(3), 1839 (2023)
62. Shereen, E., Ristea, D., Vyas, S., McFadden, S., Dwyer, M., Hicks, C., Mavroudis, V.: Sok: On closing the applicability gap in automated vulnerability detection. arXiv:2412.11194 (2024)
63. Duchène, F.: State of the art: a study of security vulnerability assessment solutions for web applications. Comput. Secur. **44**, 100–110 (2014)
64. Utting, M., Legeard, B.: Practical Model-Based Testing: A Tools Approach. Morgan Kaufmann, San Francisco, CA (2007)
65. Myers, G.J., Sandler, C., Badgett, T.: The Art of Software Testing, 3rd edn. Wiley, Hoboken, NJ (2011)
66. Huang, Y.W.: Securing web application code by static analysis and runtime protection. In: 12th International Conference on World Wide Web, pp. 40–52 (2003)
67. Antunes, N., Vieira, M.: Designing vulnerability testing tools for web services: approach, components, and tools. Int. J. Inf. Secur. **14**(5), 435–457 (2015)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.